

Final Project - Dungeon Crawler

COMP301

Dungeon Crawler - Part 1

Introduction

In this assignment, you will develop a complete 2-D dungeon crawler game that features both a robust backend and an interactive front end using JavaFX. You will implement a full-scale application following the Model-View-Controller (MVC) and Observer design patterns, ensuring a clear separation of functionality and a highly maintainable codebase.

The provided starter code includes interfaces and class stubs that lay the groundwork for the game's core functionality. Your task is to fill in the missing parts, implementing the required behaviors in the model (game logic), the view (JavaFX user interface), and the controller (input processing and state management).

Part 1 of the assignment is the backend functionality. Part 2 will focus on building the controller and user interface in JavaFX

What You Will Build

- (Part 1) Model:
Develop the core game logic and data structures. This layer will manage the dungeon's layout, rooms, the hero, monsters, items, and other game elements. The model is also responsible for maintaining game state and notifying observers about changes.
- (Part 2) Controller:
Implement the controller to act as a mediator between the view and the model. It will handle user input (such as keyboard or mouse events), translate these events into actions in the game, and update the model accordingly. This design keeps the user interface decoupled from the business logic.
- (Part 2) View (JavaFX):
Create a dynamic user interface with JavaFX that visually represents the dungeon, its inhabitants, and interactions. The view should subscribe to updates from the model, using the Observer pattern to automatically refresh the display when the game state changes.

Apprentice

In the starter code you will see a package called `[prefix].model.pieces` These are the required pieces that we will need to make our game.

Piece

Take a look at the `Piece` interface:

```
1 public interface Piece {  
2     String getName();  
3     void setPosn(Posn posn);  
4     Posn getPosn();  
5     String getResourcePath();  
6 }
```

As you can see, these are basic setters and getters of an `Object` in java. We can intuit that there will be 3 fields:

- **String name:** The name represents a name that you may need in your code.
- **String resourcePath:**
Specifies the file path to the image associated with the piece. Important: This path should be relative to the classpath (i.e., under "src/main/resources"). You may use any images for your pieces. I encourage you to spend some time brainstorming the art style, as the images you select will dictate the game's overall look and feel. Although the sample uses a pixel art style, you are free to be creative—AI-generated art is perfectly acceptable.
- **Posn posn:** (Short for "Position") This is a simple x,y ordered pair used to simplify locations.

To get you started, we have given you the **abstract** `APiece` class that initializes all of these instance variables and implements the required methods from the interface. Your job is to implement each of the provided children classes to initialize the values of the instance variables without taking them as parameters in the constructor. *Note: Posn will be initialized to null right now, since it is not yet on the board.*

Treasure

The treasure class is the only class that has any additional state information: a value. Set the value of the treasure to 50 (shiny gold doubloons) and implement the body of the `getValue()` method.

(You may test your implementations here)

Movable Piece

Take a look at the `MovablePiece` interface and the `CollisionResult` class:

```
1 public interface MovablePiece extends Piece {
2     CollisionResult collide(Piece other);
3 }

1 public class CollisionResult {
2     private final int points;
3     private final Result res;
4     public enum Result {CONTINUE, GAME_OVER, NEXT_LEVEL}

5
6     public CollisionResult(int points, Result res) {
7         this.points = points;
8         this.res = res;
9     }

10
11     public int getPoints() {
12         return points;
13     }

14
15     public Result getResults(){
16         return res;
17     }
18 }
```

Take a look at the `enum` in the `CollisionResult` object.

When certain pieces collide on the board, specific outcomes occur. For the **hero**:

1. If the hero collides with nothing (`null`), game should continue unaffected.
2. The game will continue, but the score may change if the hero collides with a piece of treasure.
3. The game will end if the hero collides with an enemy.
4. The hero will move to the next level if they collide with the exit.

The enemy behaves differently:

1. If the enemy collides with nothing (`null`), game should continue unaffected.
2. The enemy will “eat” any treasure it finds (removing it from the board) and the game will continue.
3. The enemy can catch up with the hero and “eat” him, causing the game to end.

Additional rules:

- The enemy cannot leave the room and is therefore forbidden from colliding with the exit.
- Similarly, neither the hero nor the enemy should collide with walls.

Both the `Enemy` and `Hero` classes implement the `MovablePiece` interface. Write them to match the specifications above. *Hint: Focus on the numbered items only; you will implement these additional rules elsewhere.*

Enchanter

Board

Take a look at the `Board` interface in the `[prefix].model.board` package.

```
1 public interface Board {
2
3     void init(int enemies, int treasures, int walls);
4
5     int getWidth();
6
7     int getHeight();
8
9     Piece get(Posn posn);
10
11     CollisionResult moveHero(int drow, int dcol);
12 }
```

This is our Game Board Data Structure that holds our pieces and has encapsulated methods and manages game state. The other item in the package is the `Posn` class. This is an ordered pair of row,column coordinates. It is final so that the values are immutable.

```
1 public class Posn {
2     private final int row;
3     private final int col;
4
5     public Posn(int row, int col){
6         this.row = row;
7         this.col = col;
8     }
9
10    public int getRow(){
11        return row;
12    }
13
14    public int getCol(){
15        return col;
16    }
17
18    @Override
19    public String toString(){
20        return row + "," + col;
21    }
22 }
```

You have freedom to implement the `BoardImpl` class as you see fit. You are encouraged to implement additional private helper methods.

Class header, Instance Variables, and Constructors

Header Surprise surprise, BoardImpl should implement the Board interface.

Instance Variables You may choose to have as many instance variables as you find useful. The only one that you must have, is a properly encapsulated instance of 2-Dimensional array of Pieces called board. *Hint: As you go through this class, think about what instance variables may be useful that would prevent you from searching the entire board for a piece.*

Constructors You should have two constructors:

1. One that takes in a width and a height and initializes a **board** array with those dimensions along with any other instance variables you choose to make.
2. One that takes in a Piece[][] array and sets your instance variable with the parameter along with any other instance variables you choose to make.

At initialization, the board is empty (full of null pieces).

(Optional) toString Consider writing a toString method for board so that you can use print statements to help debug your code. It's really helpful, but not actually required.

init(int enemies, int treasures, int walls) We will have to set the board every level, with a different number of pieces each time. The method takes in the number of **Enemy** pieces, **Treasure** pieces, and **Wall** pieces. In addition, we know that there is one **Hero** and one **Exit**.

init should first clear the board of any pieces. Then, for each required piece, it should choose a random empty spot on the board, and place it.

In addition, if the number of required pieces exceeds the number of available spots on the board, the method should throw an **IllegalArgumentException**.

Getters

The board has three getter methods:

```
1 int getWidth();
2
3 int getHeight();
4
5 Piece get(Posn posn);
```

- **getWidth()**: Should return the number of columns (horizontal space)
- **getHeight()**: should return the number of rows (vertical space)
- **get(Posn posn)**: get the pieces at a particular Posn coordinate in the **board** and return it.

Movement

Hero

- **moveHero(int drow, int dcol)**: The MovablePieces will be able to move UP,DOWN, LEFT, RIGHT from its current position. The inputs **drow** and **dcol** should be the delta between the current position and the new position. For example, if the hero is at (2,3) going UP would be **moveHero(-1,0)**.
- The hero has restrictions on where it can move:
 - The hero cannot move off the bounds of the board.
 - The hero cannot move where a wall is currently placed.

If the move is illegal, the hero should remain in its current position. Otherwise, it can move into the new spot and `collide()` with any piece that might be there.

Once the hero has moved, all enemies get a chance to move. (Enemies do not move if the `Hero` action was illegal)

Once all pieces have moved, the method should return a `CollisionResult` that represents the state of the game. **BEWARE** there is some order of precedence here.

For example:

- A `Hero` can make it to the exit, in which case, the `Enemy` should not be able to perform another action. The `CollisionResult` should be `Result.NEXT_LEVEL`.
- A `Hero` can get a treasure, thus increasing the score, and then immediately die by an `Enemy`. In which case, the `CollisionResult` would need both the score from the Hero's action, and the `Result.GAME_OVER` from the Enemy's action.
- If gameplay should continue after everyone has moved, the `CollisionResult` should be `Result.CONTINUE`

Enemy Movement

For now, our enemies are going to be dumb. Every enemy should pick a random direction to go. However, they have more restrictions on its movement than a `Hero`.

- `Enemy` cannot move off the bounds of the board.
- `Enemy` cannot move where a `Wall` is currently placed.
- `Enemy` cannot move where an `Exit` is currently placed.
- `Enemy` cannot collide with another `Enemy`.

If an enemy is trapped, it should just skip its turn.

Other Movement Considerations

- The piece has a reference to its current location as an instance variable. Therefore, whenever you move it, you should update the instance variable.
- Object-based arrays store memory pointers. That means if you place the `Piece` a second time, it may show up twice on your board. You must remove the old reference when you move the piece.

Sorcerer

Model, Observer, and Subject

The last part will focus on completing the model that will hold our entire game state, including scores. Take a look at the Model interface:

```
1 public interface Model extends Subject {
2     enum STATUS { END_GAME, IN_PROGRESS }
3
4     int getWidth();
5     int getHeight();
6
7     Piece get(Posn p);
8
9     int getCurScore();
10    int getHighScore();
11    int getLevel();
12
13    //Change status from IN_PROGRESS and END_GAME
14    STATUS getStatus();
15    void startGame();
16    void endGame();
17
18    void moveUp();
19    void moveDown();
20    void moveLeft();
21    void moveRight();
22 }
```

There is an enumerated status at the top that will track whether there is an active game. In addition to the methods here, notice that `Model extends Subject` which adds one more additional method: `void addObserver(Observer o)`.

Header, Instance Variables, and Constructors

- Surprise surprise, our `ModelImpl` class needs to implement `Model`.
- You can see 7 getter methods, 2 methods that change the status, and 4 movement methods. You can intuit that there needs to be instance variables for a `Board` (to handle `getWidth`, `getHeight`, `get`), an `int curScore`, an `int highScore`, an `Status status`, `int level`, and a `List of Observers` (for the `Subject` interface).
- Write a constructor that takes in an `int width` and `int height`, and creates a new board, and initializes all instance variables to their pregame state. All ints are 0 ,and the `STATUS` should be `END_GAME`.
- Write another constructor that takes in a `Board` object and initializes all instance variables to their pregame state.

Getters

Implement all getters. Delegate to the `Board` when appropriate:

- `int getWidth()`
- `int getHeight()`
- `Piece get(Posn p)`
- `int getCurScore()`
- `int getHighScore()`
- `int getLevel()`
- `int getStatus()`

Status Methods

startGame() This is where we initialize our board and start the game logic. To begin, change the status of the game to `IN_PROGRESS`. Make sure that the current score is 0, and that the level resets to 1. Every time the hero reaches an exit, the board will be re initialized and the level will increment by 1. The number of `Enemy` pieces will be equal to $(\text{level} + 1)$. So the first room will have 2`Enemy` pieces, the second room will have 3, etc. You must have **at least 2 Treasure** pieces and 2 `Wall` pieces. However, you have freedom to decide what numbers you prefer for your game.

This means that even a perfect game will end when there are more pieces than spots on the board. When this happens, be sure to end the game.

endGame() Change the status to `END_GAME` and check to see if there is a new high score.

Movement

Next, our model specifies the actions that the user can take. In this case, moving our `Hero` up, down, left, or right. Using our `board.moveHero()` method, move the hero in the correct direction, then check the `CollisionResult` to see if the game state needs to be updated accordingly. If there are points in the `CollisionResult`, update the score. If it returns a `CollisionResult.Result.NEXT_LEVEL`, reinitialize the board at the next level. If it returns `CollisionResult.Result.GAME_OVER`, you should end the game.

Observer Pattern

Finally, we have all the pieces to implement an Observer pattern.

1. Implement the `addObserver()` method so that `Observers` can subscribe to changes in the model.
2. Implement the private `notifyObservers()` method that calls the `update()` method specified by the `Observer` interface.
3. Go through the model again. This time, whenever the game state changes, notify observers of the change.

Congrats! You've finished the backend of your dungeon crawler game.